

Lab 04 - Implementing Design Patterns

In this lab work you will manager your codes follow the given design patterns.

Adaptor pattern

The use cases that one data object is displayed in several places with different formats. We have structure Book with cover image, title, author(s), number of pages, publisher, edition number, and ISBN. The book will be displayed in the following formats:

- **Summary format:** The display is on home page where less text is better; the display is:

```
-----
| Cover image (2:1) | | Cover image2(2:1) |
| Title...         Pages | | Title2...         Pages2 |
-----
| Cover image3(2:1) | | Cover image4(2:1) |
| Title3...         Pages3 | | Title4...         Pages4 |
-----
| Cover image5(2:1) | | Cover image6(2:1) |
| Title5...         Pages5 | | Title6...         Pages6 |
-----
```

- **Details format:** The display is on one book details information; the display is:

```
-----
|                               |
|           Cover image (1:2)  |
|                               |
-----
| Title...                     Pages |
| By ____Author(s)____         |
| ISBN: __ISBN__               |
| Edition                      |
| Published by ____Publisher____ |
-----
```

- **Table row:** The display is one book a line; the display is:

No.	Title	ISBN	Author(s)	Edition	Publisher
1	Title 1	0000000000	Author1	1	Publisher1
2	Title 2	0000000001	Author2	2	Publisher2
3	Title 3	0000000002	Author3	3	Publisher3

MVC Pattern

Add to the first task options **Add Book**, **Edit Book**, **Delete Book**. Then, implement the actions follow the MVC pattern:

- **Add Book** chosen
 - Show add book form (let user fill the form and submit to BookController)
 - BookController will validate book data and then
 - BookController ask BookModel to store book data to file
 - BookController will return result success (0).
- **Edit Book** chosen
 - Show edit book form (let user edit the form data and submit to BookController)
 - BookController will validate book data and then
 - BookController ask BookModel to update book data in file
 - BookController will return result success (0).
- **Delete Book** chosen
 - Ask user to confirm if user really want to delete and then
 - Submit book id to BookController for deletion
 - BookController ask BookModel to check if book exists
 - If book exists BookController will ask BookModel to delete that book from file.

Submit as link to your Gitlab project.

Prototype

Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype. Pattern has creational purpose and deals with object relationships, which are more dynamic. The pattern hides the complexities of making new instances from the client.

When to use

when the classes to instantiate are specified at run-time to avoid building a class hierarchy of factories that parallels the class hierarchy of products when instances of a class can have one of only a few different combinations of state

Prototype pattern

Follow the Github sample prototype pattern: <https://github.com/srengty/design-patterns/tree/master/CreationalPatterns/prototype> Create a class called Student that represent student of International program class. Create the prototype and then, add clone method to create a copy of the prototype with some default values.

Usage of Prototype pattern

Supplement to previous task, create a menu that let user to listing, add, update, remove student(s) from group of students.

```
----- Menu -----  
1. List students  
2. Add a new student (use prototype)  
3. Create multiple students (use prototype)  
4. Update a student by index  
5. Remove a student by index  
6. Remove all students  
0. Exit  
-----  
> Choose an option:
```